and then decrypt it using a `DirectDecrypter` initialized with the AES key, as shown in listing 6.7. If the authentication tag validation fails during decryption, then the library will throw an exception. To further reduce the possibility of padding oracle attacks in CBC mode, you should never return any details about why decryption failed to the user, so just return an empty `Optional` here as if no token had been supplied. You can log the exception details to a debug log that is only accessible to system administrators if you wish. Once the JWT has been decrypted, you can extract and validate the claims from the JWT. Open JwtTokenStore.java in your editor again and implement the read method as in listing 6.7.

Listing 6.7 The JWT read method

```
@Override
public Optional<Token> read(Request request, String tokenId)
{
    try {
        var jwt = EncryptedJWT.parse(tokenId);    #A

        var decryptor = new DirectDecrypter(encKey);    #B
        jwt.decrypt(decryptor);    #B

        var claims = jwt.getJWTClaimsSet();
        if
(!claims.getAudience().contains("https://localhost:4567")) {
            return Optional.empty();
        }
        var expiry =
claims.getExpirationTime().toInstant();    #C
        var subject = claims.getSubject();    #C
        var token = new Token(expiry, subject);    #C
        var ignore = Set.of("exp", "sub", "aud");    #C
```